

```

# BLOB Principe - Protocole E - Test aux limites Colab
# Trace temporelle dense pour caracteriser la croissance n_tri(t)
# a differentes echelles N
#
# Question scientifique :
#   Le taux dn_tri/dt depend-il de N ?
#   Y a-t-il saturation tardive ou croissance perpetuelle ?
#
# Strategie : 3 tailles N, runs longs avec trace dense
# Optimisation : sauvegarde incrementale apres chaque seed
#                 pour ne RIEN perdre si Colab se deconnecte

import sys
import os

# =====
# AUTO-TEST CORRUPTION
# =====

def auto_check():
    print("=" * 70)
    print("  AUTO-TEST CORRUPTION CARACTERES")
    print("=" * 70)
    try:
        if hasattr(sys, 'argv') and len(sys.argv) > 0 and os.path.exists(sys.argv
            source_file = sys.argv[0]
        else:
            source_file = __file__ if '__file__' in globals() else None
        if source_file and os.path.exists(source_file):
            with open(source_file, 'r', encoding='utf-8') as f:
                content = f.read()
            dangerous_chars = {
                chr(8216): "U+2018", chr(8217): "U+2019",
                chr(8220): "U+201C", chr(8221): "U+201D",
                chr(8211): "U+2013", chr(8212): "U+2014",
            }
            found = [(name, content.count(c)) for c, name in dangerous_chars.item
            if found:
                print("ERREUR caracteres corrompus :", found)
                return False
            star = chr(42)
            bad_init = star + star + "init" + star + star
            if content.count(bad_init) > 1:
                print("ERREUR : motif corrompu init")
                return False

```

```

        triple_back = chr(96) * 3
        if content.count(triple_back) > 1:
            print("ERREUR : Markdown detecte")
            return False
        print("OK : pas de corruption")
    else:
        print("Note : auto-test partiel")
except Exception as e:
    print("Note : " + str(e))
print("=" * 70)
return True

if not auto_check():
    sys.exit(1)

# =====
# IMPORTS
# =====
import numpy as np
import time
import json
import gc
from collections import defaultdict

# =====
# Parametres canoniques
# =====
KAPPA_0 = 8
BETA = 0.30
TAU = 0.50

OUTPUT_DIR = '/content/blob_E_results'
os.makedirs(OUTPUT_DIR, exist_ok=True)

# =====
# Classe BlobGraph (optimisee memoire pour grandes N)
# =====
class BlobGraph:
    def __init__(self, N, seed, is_null=False):
        self.N = N
        self.is_null = is_null
        self.rng = np.random.default_rng(seed)
        self.E_target = int(N * KAPPA_0 / 2)
        self.adj = [set() for _ in range(N)]

```

```

self.edge_list = []
self._init_graph()
self.deg = np.array([len(self.adj[i]) for i in range(N)], dtype=np.int32)
self.n_tri = self._count_triangles()
self.S_curr = self._compute_S(self.n_tri)

def _init_graph(self):
    for i in range(self.N):
        u = i
        v = (i + 1) % self.N
        if v not in self.adj[u]:
            self.adj[u].add(v)
            self.adj[v].add(u)
            self.edge_list.append((min(u, v), max(u, v)))
    attempts = 0
    max_attempts = self.E_target * 10
    while len(self.edge_list) < self.E_target and attempts < max_attempts:
        u = int(self.rng.integers(self.N))
        v = int(self.rng.integers(self.N))
        attempts += 1
        if u != v and v not in self.adj[u]:
            self.adj[u].add(v)
            self.adj[v].add(u)
            self.edge_list.append((min(u, v), max(u, v)))

def _count_triangles(self):
    tri = sum(len(self.adj[u] & self.adj[v]) for u, v in self.edge_list)
    return tri // 3

def _compute_S(self, n_tri_val):
    mu = float(self.deg.mean())
    D = float(self.deg.std()) / mu if mu > 0 else 0.0
    return np.log(1 + n_tri_val) - BETA * D

def step_blob(self):
    idx = self.rng.integers(len(self.edge_list))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False
    u_add, v_add = min(u_add, v_add), max(u_add, v_add)
    if (u_add, v_add) == (u_rm, v_rm):
        return False
    dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
    self.adj[u_rm].discard(v_rm)
    self.adj[v_rm].discard(u_rm)

```

```

self.deg[u_rm] -= 1
self.deg[v_rm] -= 1
dt_add = len(self.adj[u_add] & self.adj[v_add])
self.adj[u_add].add(v_add)
self.adj[v_add].add(u_add)
self.deg[u_add] += 1
self.deg[v_add] += 1
new_n_tri = self.n_tri - dt_rm + dt_add
if new_n_tri < 0:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False
new_S = self._compute_S(new_n_tri)
log_alpha = self.N * (new_S - self.S_curr) / TAU
if log_alpha >= 0:
    accepted = True
else:
    accepted = self.rng.random() < np.exp(max(log_alpha, -500))
if accepted:
    self.edge_list[idx] = (u_add, v_add)
    self.n_tri = new_n_tri
    self.S_curr = new_S
    return True
else:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False

def step_null(self):
    idx = self.rng.integers(len(self.edge_list))
    u_rm, v_rm = self.edge_list[idx]
    u_add = int(self.rng.integers(self.N))
    v_add = int(self.rng.integers(self.N))
    if u_add == v_add or v_add in self.adj[u_add]:
        return False

```

```

u_add, v_add = min(u_add, v_add), max(u_add, v_add)
if (u_add, v_add) == (u_rm, v_rm):
    return False
dt_rm = len(self.adj[u_rm] & self.adj[v_rm])
self.adj[u_rm].discard(v_rm)
self.adj[v_rm].discard(u_rm)
self.deg[u_rm] -= 1
self.deg[v_rm] -= 1
dt_add = len(self.adj[u_add] & self.adj[v_add])
self.adj[u_add].add(v_add)
self.adj[v_add].add(u_add)
self.deg[u_add] += 1
self.deg[v_add] += 1
new_n_tri = self.n_tri - dt_rm + dt_add
if new_n_tri < 0:
    self.adj[u_add].discard(v_add)
    self.adj[v_add].discard(u_add)
    self.deg[u_add] -= 1
    self.deg[v_add] -= 1
    self.adj[u_rm].add(v_rm)
    self.adj[v_rm].add(u_rm)
    self.deg[u_rm] += 1
    self.deg[v_rm] += 1
    return False
self.edge_list[idx] = (u_add, v_add)
self.n_tri = new_n_tri
self.S_curr = self._compute_S(new_n_tri)
return True

```

```

# =====
# RUN AVEC TRACE DENSE + SAUVEGARDE INCREMENTALE
# =====
def run_with_trace_save(N, seed, total_steps, is_null, label, save_every_n_sample
    """
    Trace dense + sauvegarde JSON apres chaque batch d echantillons.
    Si le run plante, on a quand meme la trace partielle.
    """
    t_start = time.time()
    type_label = 'NULL' if is_null else 'BLOB'
    log_id = ' [' + label + ' ' + type_label + ' N=' + str(N) + ' s=' + str(seed)
    print(log_id + ' Init ' + str(total_steps) + ' steps...', flush=True)

    sim = BlobGraph(N=N, seed=seed, is_null=is_null)
    step_func = sim.step_null if is_null else sim.step_blob

    print(log_id + ' Init OK |E|=' + str(len(sim.edge_list))

```

```

        + ' n_tri=' + str(sim.n_tri)
        + ' init_t=' + '{:.0f}'.format(time.time() - t_start) + 's', flush=True

n_samples = 200
sample_steps = sorted(set([
    max(1, int(np.exp(np.log(total_steps) * i / n_samples)))
    for i in range(1, n_samples + 1)
]))

trace = []
acc = 0
next_idx = 0
save_path = OUTPUT_DIR + '/E_' + label + '_' + type_label + '_N' + str(N) + '

for i in range(total_steps):
    if step_func():
        acc += 1
    if next_idx < len(sample_steps) and (i + 1) >= sample_steps[next_idx]:
        sample = {
            'step': i + 1,
            'n_tri': sim.n_tri,
            'acc_cumul': acc / (i + 1),
            'time_elapsed': time.time() - t_start
        }
        trace.append(sample)
        next_idx += 1

# Sauvegarde incrementale
if next_idx % save_every_n_samples == 0:
    partial = {
        'N': N, 'seed': seed, 'type': type_label, 'label': label,
        'total_steps_planned': total_steps,
        'partial': True,
        'trace': trace
    }
    try:
        with open(save_path, 'w') as f:
            json.dump(partial, f)
    except Exception as e:
        print(log_id + ' WARN save failed: ' + str(e), flush=True)

# Print intermediaire
print('    step ' + str(i+1) + '/' + str(total_steps)
      + ' n_tri=' + str(sim.n_tri)
      + ' acc=' + '{:.3f}'.format(acc/(i+1))
      + ' t=' + '{:.0f}'.format(time.time()-t_start) + 's', flush

```

```

total_time = time.time() - t_start
final_acc = acc / total_steps if total_steps > 0 else 0

# Stats finales
last_q = trace[3 * len(trace) // 4:] if len(trace) >= 4 else trace
n_tri_q4_mean = np.mean([t['n_tri'] for t in last_q])
n_tri_q4_std = np.std([t['n_tri'] for t in last_q])

# Estimation du taux de croissance dn_tri / dt
# Sur la derniere moitie, par regression lineaire
half_trace = trace[len(trace)//2:]
if len(half_trace) >= 2:
    steps_half = np.array([t['step'] for t in half_trace])
    ntri_half = np.array([t['n_tri'] for t in half_trace])
    # Regression lineaire
    slope, intercept = np.polyfit(steps_half, ntri_half, 1)
    # slope = dn_tri / dt en triangles par pas MCMC
    # Inverse : pas par triangle
    steps_per_tri = 1.0 / slope if slope > 0 else float('inf')
else:
    slope = 0
    steps_per_tri = float('inf')

final = {
    'N': N, 'seed': seed, 'type': type_label, 'label': label,
    'total_steps_planned': total_steps,
    'total_steps_done': total_steps,
    'partial': False,
    'n_tri_final': int(sim.n_tri),
    'n_tri_q4_mean': float(n_tri_q4_mean),
    'n_tri_q4_std': float(n_tri_q4_std),
    'acc_global': float(final_acc),
    'time_total_sec': float(total_time),
    'growth_slope_per_step': float(slope),
    'steps_per_triangle': float(steps_per_tri) if steps_per_tri != float('inf')
    'trace': trace
}

try:
    with open(save_path, 'w') as f:
        json.dump(final, f)
except Exception as e:
    print(log_id + ' WARN final save failed: ' + str(e), flush=True)

print(log_id + ' DONE. n_tri_final=' + str(sim.n_tri)
      + ' n_tri_q4=' + '{:.0f}'.format(n_tri_q4_mean)
      + ' growth=' + '{:.5f}'.format(slope) + ' tri/step'

```

```

        + ' acc=' + '{:.3f}'.format(final_acc)
        + ' t=' + '{:.0f}'.format(total_time) + 's', flush=True)

# Liberation memoire
del sim
gc.collect()

return final

# =====
# MAIN PROTOCOLE E
# =====
print('')
print('=' * 75)
print('  PROTOCOLE E : Test aux limites Colab')
print('=' * 75)
print('  Question : dn_tri/dt depend-il de N ?')
print('  Strategie : trace dense aux 3 tailles, sauvegarde incrementale')
print('  Tout est sauvegarde dans : ' + OUTPUT_DIR)
print('=' * 75)

# Configuration : runs adaptes au temps Colab
RUNS = [
    # N, total_steps, is_null, n_seeds, label
    (250000, 7500000, False, 2, 'L1'),      # 250k - 30N - 2 seeds Blob
    (250000, 7500000, True, 1, 'L1'),       # 250k - 30N - 1 seed Null
    (500000, 10000000, False, 1, 'L2'),     # 500k - 20N - 1 seed Blob
    (500000, 10000000, True, 1, 'L2'),      # 500k - 20N - 1 seed Null
    (1000000, 5000000, False, 1, 'L3'),     # 1M - 5N - 1 seed Blob
    (1000000, 5000000, True, 1, 'L3'),      # 1M - 5N - 1 seed Null
]

# Calcul d estimation
total_estimated_seconds = 0
print('')
print('  Configuration des runs :')
print('  ' + 'label'.ljust(6) + 'N'.ljust(10) + 'steps'.ljust(12) + 'type'.ljust(
    + 'seeds'.ljust(8) + 'temps_estime')
for N, steps, is_null, n_seeds, label in RUNS:
    # Estimation : ~10 microsec/pas a N=50k, scaling lineaire en N (a vue)
    # Plus realistement : O(deg) par pas, deg=8, donc constant
    # Mais grandes N : alloc plus longue. ~1M pas/sec sur Colab CPU
    sec_per_seed = steps / 1e6 * 60 # ~60s par million de pas
    seed_time = sec_per_seed * n_seeds
    total_estimated_seconds += seed_time
    print('  ' + label.ljust(6) + str(N).ljust(10) + str(steps).ljust(12)

```

```

        + ('NULL' if is_null else 'BLOB').ljust(8) + str(n_seeds).ljust(8)
        + '{:.0f}'.format(seed_time / 60) + ' min')

print('')
print('  Temps total estime : ' + '{:.0f}'.format(total_estimated_seconds / 60) +
      + ' (' + '{:.1f}'.format(total_estimated_seconds / 3600) + ' h)')
print('')
print('  IMPORTANT : si Colab se deconnecte, les fichiers JSON dans')
print(' ' + OUTPUT_DIR + ' contiennent les traces partielles')
print('=' * 75)

t_global = time.time()
all_results = []

for N, steps, is_null, n_seeds, label in RUNS:
    print('')
    print('--- ' + label + ' : N=' + str(N) + ', ' + str(steps)
          + ' steps, ' + ('NULL' if is_null else 'BLOB') + ', '
          + str(n_seeds) + ' seeds ---')
    for seed in range(n_seeds):
        try:
            r = run_with_trace_save(N, seed if not is_null else 100 + seed,
                                    steps, is_null, label)

            all_results.append(r)
        except Exception as e:
            print('  ERREUR run : ' + str(e), flush=True)
            print('  Continuation avec le suivant...', flush=True)
            import traceback
            traceback.print_exc()

global_time = time.time() - t_global

# =====
# ANALYSE FINALE
# =====
print('')
print('=' * 75)
print('  ANALYSE FINALE')
print('=' * 75)
print('  Temps total reel : ' + '{:.1f}'.format(global_time / 60) + ' min')
print('')

# Regrouper par (label, N, type)
print('  Synthese :')
print(' ' + 'label'.ljust(6) + 'N'.ljust(10) + 'type'.ljust(8)
      + 'n_tri_final'.ljust(14) + 'n_tri_q4'.ljust(12))

```

```

        + 'growth_per_step'.ljust(18) + 'acc')
print(' ' + '-' * 80)
for r in all_results:
    print(' ' + r['label'].ljust(6) + str(r['N']).ljust(10)
          + r['type'].ljust(8)
          + str(r['n_tri_final']).ljust(14)
          + '{:.0f}'.format(r['n_tri_q4_mean']).ljust(12)
          + '{:.6f}'.format(r['growth_slope_per_step']).ljust(18)
          + '{:.3f}'.format(r['acc_global'])))

# Question scientifique : dn_tri/dt depend-il de N ?
print('')
print(' Croissance dn_tri/dt par taille (Blob seulement) :')
blob_results = [r for r in all_results if r['type'] == 'BLOB']
for r in blob_results:
    n_per_tri = 1.0 / r['growth_slope_per_step'] if r['growth_slope_per_step'] >
    print('      N=' + str(r['N']) + ' : '
          + '{:.6f}'.format(r['growth_slope_per_step']) + ' tri/step '
          + '({:.0f}'.format(n_per_tri) + ' steps par triangle)')

if len(blob_results) >= 2:
    Ns = sorted(set([r['N'] for r in blob_results]))
    if len(Ns) >= 2:
        # Une moyenne par N
        rates = []
        for N in Ns:
            rs = [r for r in blob_results if r['N'] == N]
            mean_rate = np.mean([r['growth_slope_per_step'] for r in rs])
            rates.append(mean_rate)

        # Loi rate ~ N^alpha
        log_N = np.log(Ns)
        log_rate = np.log([max(r, 1e-15) for r in rates])
        if len(Ns) >= 2:
            slope_alpha, intercept_alpha = np.polyfit(log_N, log_rate, 1)
            print('')
            print(' Loi : dn_tri/dt ~ N^' + '{:.3f}'.format(slope_alpha))

            if abs(slope_alpha) < 0.1:
                print(' -> Taux INDEPENDANT de N : croissance intrinseque univer
            elif slope_alpha > 0.5:
                print(' -> Taux croit avec N : phenomene cooperatif a grande ech
            elif slope_alpha < -0.5:
                print(' -> Taux decroit avec N : effet de dilution')

# Recap fichiers
print('')

```

```
print(' Fichiers JSON sauvegardes :')
for f in sorted(os.listdir(OUTPUT_DIR)):
    print('      ' + OUTPUT_DIR + '/' + f)

print('')
print('=' * 75)
print('  FIN PROTOCOLE E')
print('=' * 75)
```